

5 Levels of @claude-code-guide

Copy-paste prompts to unlock Claude Code's
most underrated built-in feature

Ask > Diagnose > Understand > Build > Self-Improve

What is @claude-code-guide?

Claude Code has a built-in sub-agent that most people don't know exists. It's called **claude-code-guide**, and its job is to read Claude Code's own documentation and act on what it finds.

Instead of Googling how a feature works, you tag **@claude-code-guide** in your prompt and ask in plain English. Claude looks up the docs, understands the answer, and either explains it to you or executes it directly.

This guide gives you 5 levels of prompts, each more powerful than the last. Copy them, paste them into Claude Code, and watch what happens.

How to use these prompts:

- Open Claude Code in any project folder
- Copy the prompt from the dark box
- Paste it into Claude Code and press Enter
- The @claude-code-guide tag tells Claude to search its own docs before answering
- Each level builds on the last, so go in order the first time

LEVEL 1

Just Ask

The simplest use. You have a question about a Claude Code feature but you don't want to leave the terminal to look it up. By tagging **@claude-code-guide**, you're telling Claude to search its own documentation before answering. This means you get the real answer from the actual docs, not a guess from the model's training data.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide how do I change my model mid-session? Like if I'm using Sonnet and I want to switch to Opus for a harder task, what's the command?
```

How this works with the sub-agent:

Claude spawns the guide sub-agent, which searches the Claude Code documentation for model switching. It finds the `/model` command, explains the available models (Opus 4.6, Sonnet 4.6, Haiku 4.5), and tells you when to use each one. You get the real answer without leaving the terminal.

LEVEL 2

Diagnose a Problem

Now you're combining two things: a file from your project AND the guide. By tagging both **@CLAUDE.md** (your file) and **@claude-code-guide** (the docs), you're asking Claude to compare what you have against what best practices say you should have. It reads your config, reads the docs, and tells you what's missing.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide look at @CLAUDE.md - I keep having to repeat myself to Claude about how I want my code formatted and where to save things. Why isn't this CLAUDE.md helping me? What's missing?
```

How this works with the sub-agent:

Claude reads your CLAUDE.md file, then searches the docs for CLAUDE.md best practices. It compares the two and gives you a gap analysis: what you're missing, what you should add, and why your current file isn't giving Claude enough context to help you consistently.

LEVEL 3a

Learn the Landscape

Before you build any automation, you need to know what's possible. This prompt asks the guide to give you the full menu of hook types available in Claude Code. Hooks are automations that trigger when Claude does something (edits a file, starts a session, etc). By asking the guide first, you learn what levers exist before you pull any of them.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide what types of hooks are available in Claude Code? I want to understand what actions I can attach automations to. Like, can I trigger something every time a file is created, or when a session starts, or when Claude edits something? Walk me through what's possible.
```

How this works with the sub-agent:

The guide searches the documentation for hook types and returns the full list: PreToolUse, PostToolUse, Notification, SessionStart, Stop, and more. For each one, it explains when it fires and gives examples. You now have the complete picture of what you can automate.

LEVEL 3b

Diagnose an Existing Hook

Same pattern as Level 2, but now applied to hooks. You tag your settings file AND the guide, then ask why your hook isn't working properly. Claude reads your config, reads the docs on how hooks should be structured, and tells you exactly what's wrong.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide look at my current hook setup in @.claude/settings.json - I have a hook that's supposed to archive my plan files but it's just dumping them into past-plans/ with random names. What's wrong with this setup and how should it actually work?
```

How this works with the sub-agent:

Claude reads your settings.json hook configuration, then searches the docs for the correct hook syntax and behavior. It explains what your hook is actually doing (copying with no rename logic) versus what it should do, and suggests the fix.

LEVEL 4

Build a Custom Command

This is where it gets powerful. You describe a workflow you want in plain English, and the guide looks up how custom slash commands work, then builds one for you. Custom commands are markdown files that become one-word shortcuts. The guide handles the file format, the directory structure, and the prompt engineering. You just describe the outcome.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide I want to create a custom slash command called /standup that looks at what I worked on in my last session and writes a 3-bullet summary I can copy and paste into my team's standup chat. Show me how to create the file for this.
```

How this works with the sub-agent:

Claude searches the docs for custom slash command syntax, learns the file format and directory conventions (.claude/commands/), then creates the standup.md file with the right structure. You can immediately type /standup and it works.

LEVEL 5

Self-Improving Automation

The finale. You're pointing Claude at its own broken automation and asking it to fix itself. It reads the docs to understand how hooks should work, reads your existing broken config, rewrites the hook with smarter logic, AND cleans up the mess the old version created. This is Claude using its own documentation to upgrade its own infrastructure.

PROMPT (copy and paste into Claude Code):

```
@claude-code-guide look at @.claude/settings.json and the past-plans/ folder. My archive hook saves plans with random names like "mossy-napping-hopper.md" which makes it impossible to find anything later.
```

```
I want you to fix this hook so that:
```

1. Archived plans get named with the date and a short description of what the plan was about (like 2026-03-13-add-user-auth.md)
2. It reads the first few lines of the plan file to figure out the description
3. It organizes them into past-plans/ sorted by date

```
Also rename the existing files in past-plans/ to follow this new naming convention based on their content.
```

How this works with the sub-agent:

Claude reads the hook documentation, reads your current broken config, rewrites the hook with intelligent naming logic that extracts the plan topic from file content, then goes through every existing file in past-plans/, reads each one, and renames them from garbage names to clean, descriptive, dated filenames. One prompt transformed the entire system.

Tips for Using @claude-code-guide

- **Always tag the file you're asking about.** The guide is smarter when it can see both the docs AND your actual config. @settings.json + @claude-code-guide is more powerful than @claude-code-guide alone.
- **Use it when Claude seems unsure.** If Claude hesitates or gives a vague answer about one of its own features, tag the guide. It forces a doc lookup instead of a guess.
- **It works for any Claude Code feature.** Hooks, slash commands, CLAUDE.md, subagents, MCP servers, headless mode, permissions. If it's a Claude Code feature, the guide can look it up.
- **You can chain it with other sub-agents.** Ask the guide how a feature works, then ask the Explore sub-agent to find where that feature is configured in your project.
- **It's built in. No setup required.** You don't need to install anything. Just type @claude-code-guide in any prompt and it activates.

Made by Early AI-dopters. Learn more at earlyaidopters.com